

 c0a2509236 / ProjExD_Group06





[Code](#) [Issues](#) [Pull requests](#) [Actions](#) [Projects](#) [Wiki](#) [More](#)

  C0A25092/featur... [ProjExD_Group06](#) / [group06.py](#)



 

 c0a2509236 リザルト画面実装完了

da9691a · 23 minutes ago 

[Code](#) [Blame](#) 210 lines (169 loc) · 9.57 KB

[Raw](#)     

```
1  import sys
2  import os
3  import pygame
4
5
6  # 実行ファイルのディレクトリにカレントディレクトリを変更（素材やスコアファイルの読み込みエラー防止）
7  os.chdir(os.path.dirname(os.path.abspath(__file__)))
8
9  # --- 各自のモジュールをインポートする準備（後ほど合流） ---
10 # import player      # B君: プレイヤー（カゴ）担当
11 # import item         # C君: 落ちてくる物体（アイテム）担当
12 # import judge        # D君: 当たり判定&スコア担当
13 # import ui           # E君: UI（文字表示）&BGM担当
14 # import assets       # F君: グラフィック（素材・演出）担当
15 # import ranking      # G君: リザルト画面（スコア記録・ランキング）担当
16
17 # 定数定義（最初の1時間で全員で決める内容の例）
18 SCREEN_WIDTH = 800
19 SCREEN_HEIGHT = 600
20 FPS = 60
21
22 # 色の定義（RGB値）
23 WHITE = (255, 255, 255)
24 BLACK = (0, 0, 0)
25 BLUE = (0, 100, 255)
26 RED = (255, 50, 50)
27 YELLOW = (255, 215, 0)
28 ORANGE = (255, 120, 0) #追加
29 CYAN = (255, 122, 0) #追加
30
31 FONT_FILE = "PixelMplus12-Regular.ttf" #追加G 使用するドットフォントのファイル名
32 HIGHSCORE_FILE = "highscore.txt"
33
34 def load_highscore():
35     """ゲーム起動時に最高スコアを読み込む関数"""
36     if os.path.exists(HIGHSCORE_FILE):
37         with open(HIGHSCORE_FILE, "r") as f:
38             try:
39                 return int(f.read())
40             except ValueError:
```

```

41         return 0
42     return 0
43
44     def check_and_save_highscore(current_score, current_highscore):
45         """今回のスコアがハイスコアを超えていたらファイルに保存する関数"""
46         if current_score > current_highscore:
47             with open(HIGHSCORE_FILE, "w") as f:
48                 f.write(str(current_score))
49             return current_score # 新しいハイスコアを返す
50         return current_highscore # 更新されなければ今のハイスコアをそのまま返す
51
52
53     def main():
54         # Pygameの初期化
55         pygame.init()
56         screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
57         pygame.display.set_caption("落ち物キャッチゲーム")
58         clock = pygame.time.Clock()
59
60         # ゲームの状態管理用変数
61         # "TITLE": タイトル画面, "PLAY": ゲーム中, "GAMEOVER": ゲームオーバー (リザルト) 画面
62         game_state = "TITLE"
63
64         # --- [D君・G君合流用変数] スコア管理の土台 ---
65         current_score = 0 # 今回のスコア (D君がゲーム中に加算する)
66         high_score = 0 # 最高スコア (G君がファイルから読み込む)
67         high_score = load_highscore() #ゲーム機同時に最高スコアを読み込む
68
69         # [G君の合流ポイント①: ゲーム起動時に最高スコアを読み込む]
70         # 例: high_score = ranking.load_highscore()
71
72         try: #追加G
73             font = pygame.font.Font(FONT_FILE, 48)
74             small_font = pygame.font.Font(FONT_FILE, 36)
75             large_font = pygame.font.Font(FONT_FILE, 64)
76         except Exception:
77             # 万が一フォントファイルがない場合のフォールバック (エラー落ち防止)
78             print(f"Warning: {FONT_FILE} が見つかりません。デフォルトフォントを使用します。")
79             font = pygame.font.SysFont(None, 48)
80             small_font = pygame.font.SysFont(None, 36)
81             large_font = pygame.font.SysFont(None, 64)
82
83         # フォントの用意 (E君・G君のUI表示用フォントが決まるまでの暫定)
84         #追加G タイマー用の変数準備
85         start_ticks = 0
86         time_left = 30
87
88         running = True
89         while running:
90             # FPS (フレームレート) の設定
91             clock.tick(FPS)
92
93             # =====
94             # 1. イベント処理 (キー入力や閉じるボタンの監視)
95             # =====
96             for event in pygame.event.get():
97                 if event.type == pygame.QUIT:
98                     running = False

```

```

99
100     # キーボードが押されたときの処理
101     if event.type == pygame.KEYDOWN:
102         if game_state == "TITLE":
103             if event.key == pygame.K_SPACE:
104                 # タイトル画面でスペースキーを押したらゲーム開始
105                 current_score = 0 # スコアをリセット
106                 time_left = 30
107                 start_ticks = pygame.time.get_ticks() #追加G タイマー開始
108                 game_state = "PLAY"
109
110             elif game_state == "PLAY":
111                 # [B君の合流ポイント⑥]
112                 # ゲーム中のキー入力（左右の移動など）は主にB君のモジュールに引き渡す
113                 pass
114
115             elif game_state == "GAMEOVER":
116                 if event.key == pygame.K_SPACE:
117                     # 追加G タイムオーバー画面でスペースキーを押したらタイトルに戻る
118                     game_state = "TITLE"
119
120     # =====
121     # 2. データ更新処理（ゲームロジック）
122     # =====
123     if game_state == "TITLE":
124         pass
125
126     elif game_state == "PLAY":
127         # [B君の合流ポイント⑥: プレイヤーの移動更新]
128         # [C君の 合流ポイント⑥: アイテムの落下更新]
129         # [D君の合流ポイント⑥: 当たり判定の計算と、current_scoreの加算・減算]
130         elapsed_seconds = (pygame.time.get_ticks() - start_ticks) / 1000 #追加G 残り時間の計算
131         time_left = 30 - elapsed_seconds
132
133         # 追加G タイムオーバー判定
134         if time_left <= 0:
135             time_left = 0
136             high_score = check_and_save_highscore(current_score, high_score) #ハイスコアを判定して保存
137             game_state = "GAMEOVER"
138
139
140         keys = pygame.key.get_pressed()
141         if keys[pygame.K_RETURN]:
142             current_score = 150 # テスト用にスコアが入ったと仮定
143             high_score = check_and_save_highscore(current_score, high_score)
144             game_state = "GAMEOVER"
145
146         # MVP確認用の暫定ルール（エンターキーを押したらゲーム終了・リザルトへ移動）
147         # ※本番はD君の判定で「タイムアップ」や「ライフ0」になったら切り替える
148         # [G君の合流ポイント⑥: ゲーム終了時にハイスコアを判定して保存]
149
150     elif game_state == "GAMEOVER":
151         pass
152
153     # =====
154     # 3. 描画処理（画面への表示）
155     # =====
156     screen.fill(BLACK) # 画面を黒でクリア

```

```
157
158     if game_state == "TITLE":
159         # タイトル画面の描画 (E君・F君の素材が来るまでの暫定表示)
160         # タイトル画面を (ドット風、中央揃え)
161         title_text = large_font.render("FALLING CATCH GAME", True, ORANGE)
162         start_text = font.render("Press SPACE to Start", True, WHITE)
163
164         title_rect=title_text.get_rect(center=(SCREEN_WIDTH//2,200))    #スタートタイトルの位置設定 (画面中
165         start_rect=start_text.get_rect(center=(SCREEN_WIDTH//2,400)) #スタート文字の位置設定 (画面中央x=4
166
167         # 画面に描画
168         screen.blit(title_text, title_rect)
169         screen.blit(start_text, start_rect)
170
171     elif game_state == "PLAY":
172         # [F君・B君の合流ポイント: プレイヤー (カゴ) の描画]
173         # [F君・C君の合流ポイント: アイテム (果物・爆弾) の描画]
174         # [E君の合流ポイント⑥: 画面上部への「現在のスコア」や「残り時間」の文字描画]
175         time_text = font.render(f"TIME: {int(time_left)}", True, WHITE) #追加G 残り時間の表示
176         screen.blit(time_text, (20,20))
177
178         # MVP確認用の暫定表示
179         play_text = font.render("Playing... [Enter:Finish]", True, WHITE)
180         play_rect = play_text.get_rect(center=(SCREEN_WIDTH // 2, 280))
181         screen.blit(play_text, play_rect)
182
183     elif game_state == "GAMEOVER":
184         # [G君の合流ポイント⑥: リザルト画面の描画 (A君の画面を乗っ取る)]
185         over_text = font.render("TIME OVER", True, RED)
186         over_rect = over_text.get_rect(center=(SCREEN_WIDTH // 2, 150))
187         screen.blit(over_text, over_rect)
188         # G君が作成した文字配置や、F君が作ったリザルト背景などをここで描画する
189
190         # スコアとハイスコアの表示 (G君の担当箇所)
191         score_text = small_font.render(f"YOUR SCORE: {current_score}", True, WHITE)
192         highscore_text = small_font.render(f"HI-SCORE: {high_score}", True, YELLOW)
193         score_rect = score_text.get_rect(center=(SCREEN_WIDTH // 2, 270))
194         highscore_rect = highscore_text.get_rect(center=(SCREEN_WIDTH // 2, 340))
195         screen.blit(score_text, score_rect)
196         screen.blit(highscore_text, highscore_rect)
197
198         retry_text = small_font.render("Press SPACE to Title", True, WHITE)
199         screen.blit(retry_text, (260, 450))
200
201     # 画面の更新
202     pygame.display.flip()
203
204     pygame.quit()
205     sys.exit()
206
207
208
209 if __name__ == "__main__":
210     main()
```